Week 9: Calling a model from code

AI Development Course

Your program asks a model, and you keep the key safe

Tested with the anthropic package 1.13.0 (Python 3.12.3 on Ubuntu 24.04, in a container), real calls to claude-haiku-5-5, and the deal screener's 92 offline tests, 12 October 2026. Prices: $0.10 / $0.50 per million tokens (Anthropic's pricing page that day). Steps 3 to 5 were also run in Windows (WSL) on 29 October 2026, in a signed-in session. The Console, GitHub and Macs are being tested.

Today

- Claude Pro vs API credit, tokens, and what one call costs
- Getting a key, and the key rules
- Your first call, from a scratch folder
- The settings that matter: system prompt, `max_tokens`, structured output, the model
- Why you can't set temperature any more
- One AI feature in your project, that still works without a key
- Review, Docker with `--env-file`, pull request

Claude Pro vs API credit

	Claude Pro	API credit
Pays for	Claude Code, the chat app	calls **your program** makes
Bought in	your Claude account	the **Claude Console**
Billed	monthly subscription	per token, from prepaid credit

Anthropic: "A paid Claude subscription enhances your chat experience, and the Claude API is billed separately."

The syllabus budgets about **$5** for weeks 9 to 11.

Tokens and cost

- **Input tokens:** everything you send. **Output tokens:** everything the model writes.
- `claude-haiku-5-5`: **$0.10** per million input, **$0.50** per million output

A call with 500 input and 200 output tokens (example numbers):

```
500 x $0.10 / 1,000,000  =  $0.00005
200 x $0.50 / 1,000,000  =  $0.00010
total                    =  $0.00015
```

Every reply says exactly how many tokens it used.

The key rules

1. **You** put the key in `.env`, with a text editor. Never in Claude Code, a chat, a screenshot, a commit or a PR.
2. Before committing: `git check-ignore .env` must print `.env`.
3. Your program reads the key from the environment and **never prints it**.
   Start Claude Code in a window where the key is **not** loaded.
4. Docker gets it at run time: `--env-file .env`. Never in the image.
5. Set a monthly **spend limit** before your first call.
6. A key leaked? **Delete it in the Console first**, then clean up.

Get a key (before class)

1. https://platform.claude.com: create an account
2. **Settings > Billing**: buy the smallest amount of credit, auto-reload **off**
3. **Settings > Billing > Spend limits**: set a small limit
4. **Settings > API keys > Create key**: choose a **workspace**; shown **once**, starts `sk-ant-`
5. `~/first-call/.env`, one line, no quote marks:

```
ANTHROPIC_API_KEY=sk-ant-...your key...
```

First call: set up

In your terminal (Python **3.10 or newer**: `python3 --version`)

```
cd ~/first-call
python3 -m venv .venv
source .venv/bin/activate
pip install anthropic==1.13.0
```

- `.venv`: a private box of packages for this folder
- Ubuntu says "ensurepip is not available"? `sudo apt install python3-venv`, delete only the broken `.venv` with `rm -r .venv`, try again

First call: the program

```
import anthropic

client = anthropic.Anthropic()          # reads ANTHROPIC_API_KEY itself

message = client.messages.create(
    model="claude-haiku-5-5",
    max_tokens=1000,
    messages=[{"role": "user",
               "content": "In one sentence, what is a cap rate in real estate?"}],
)

for block in message.content:           # pick text blocks: thinking may come first
    if block.type == "text":
        print(block.text)

print("Stopped because:", message.stop_reason)
print("Input tokens:", message.usage.input_tokens)
print("Output tokens:", message.usage.output_tokens)
```

Run: `set -a; . ./.env; set +a` then `python first_call.py`

The same question, three times

```
for i in 1 2 3; do python first_call.py; done
```

- The wording can change each time (our run: 56, 75 and 58 output tokens): a model picks words with some randomness
- **Temperature** used to turn that down. Not any more:
  - Haiku 5.5: "If a request includes `temperature`, it must be `1`"
  - Sonnet 5.5 and Opus 5.5 reject any non-default value
  - The package: `TypeError: ... unexpected keyword argument 'temperature'`
- So: make answers **dependable**, then **check** them

Controls that work

Control	What it does
A specific prompt	exactly what to do, how long, in what form
`system`	rules for the whole conversation
`max_tokens`	caps the answer, and the cost. Check `stop_reason`
Structured output	the answer comes back as JSON in a fixed shape
Checks in your code	is each value there, and sensible?
A person confirms	for anything that matters

Structured output

```
schema = {
    "type": "object",
    "properties": {
        "price": {"type": ["number", "null"], "description": "Asking price in dollars"},
        "bedrooms": {"type": ["number", "null"], "description": "Number of bedrooms"},
    },
    "required": ["price", "bedrooms"],
    "additionalProperties": False,
}

message = client.messages.create(
    model="claude-haiku-5-5", max_tokens=1000,
    system="Pull numbers out of real estate listings. Use null for any number the listing does not state.",
    messages=[{"role": "user", "content": "Cozy 3 bed, 2 bath bungalow, now $189,000."}],
    output_config={"format": {"type": "json_schema", "schema": schema}},
)
```

Our run: `{'price': 189000, 'bedrooms': 3}`, 330 input and 20 output tokens. It fixes the **shape**, not whether it's **right**.

Choosing a model

Model	ID	$ per million (in / out)	Speed
Haiku 5.5	`claude-haiku-5-5`	0.10 / 0.50	fastest
Sonnet 5.5	`claude-sonnet-5-5`	2 / 10	fast
Opus 5.5	`claude-opus-5-5`	4 / 20	moderate

Anthropic: "Begin implementation with Claude Haiku 5.5 ... Upgrade only if necessary for specific capability gaps."

Change one line, `model=`, to try another. Fine-tuning: not covered, not needed.

The deal screener's AI feature

`python3 -m dealscreener --listing`

1. Paste the listing text, then `done`
2. Haiku reads it with a schema for the five property numbers
3. You confirm each one:

```
Purchase price ($) [from the listing: 150000]:
```

4. Enter keeps it, or type the right number. The maths is unchanged.

No key? "No API key found. Put ANTHROPIC_API_KEY in a .env file (see the README). For now, type the numbers yourself."

When the call fails, the program carries on

What happened	What the deal screener says
401	The API key was not accepted. Check the key in your .env file.
429	Too many requests or tokens for now. Wait a minute and try again.
spend limit	Your monthly spend limit has been reached. ...
no credit	Your API credit has run out. Add credit in the Anthropic Console.
no internet	Couldn't reach Anthropic. Check your internet connection.

Each one, then: "For now, type the numbers yourself." Never the key, never a traceback.

Your feature

1. Branch `ai-feature` first
2. `git check-ignore .env` prints `.env`
3. Block Claude Code from reading it, in `.claude/settings.json`: `"deny": ["Read(./.env)"]`, then restart `claude`
4. `cp -n ~/first-call/.env .`
5. Pick one small job: pull fields out of text, sort into categories, or a one-sentence summary
6. `/superpowers:writing-plans` with the guardrails in the prompt, then `/superpowers:subagent-driven-development`
7. **No** to printing `.env`, keys in files, tests that call the real API. Never type `! cat .env`, even if Claude Code suggests it

Tests that don't spend money

- Tests use a **fake model**: no network, no credit, no key
- One **live** test, only when you ask: `RUN_LIVE_TESTS=1`
- A test for "no key"
- The deal screener: 92 tests, all offline

Review, Docker, finish

- `safe-and-private` on what changed: keys, `.env`, printing, real API in tests
- `docker run --rm semester-project ls -a`: no `.env`
- `docker run -it --rm --env-file .env semester-project`
- PR: **Files changed** shows no `.env`; Find `sk-ant-` finds only a fake
- Merge on GitHub, then pull

The assignment

Criterion	Points
Merged AI feature PR: fake-model tests with a "no key" test, package pinned, no key anywhere	4
Cost log of three real calls, worked out from the tokens	2
Screenshots: the feature working, and the no-key fallback	2
Reflection: which model and why, and how you made its answers dependable	2

Next week: embeddings and RAG, searching your own documents by meaning.